CS Integration/Capstone Project

Intelligent Engineering Monitoring System



Project Purpose

Develop a multi-language engineering monitoring platform that acquires sensor measurements through an Arduino-compatible environment, communicates with an embedded system, processes and stores the resulting data, performs quantitative analysis, and presents information through a graphical interface.

The central purpose is systems integration: combining hardware, embedded programming, application programming, databases, mathematical analysis, and visualization into one coherent system.



Core Project Objectives

- Integrate multiple programming environments

Use different technologies for tasks where each is appropriate rather than forcing one language to do everything. Integrate independently developed components into a single end-to-end system.

Primary stack:
Arduino / C or C++ → embedded hardware and sensor acquisition
Python → backend logic, data processing, automation, and system coordination
SQL → persistent data storage and querying
MATLAB → engineering/statistical analysis and visualization
Python GUI framework → user-facing monitoring dashboard

- Acquire real physical data

Use an Arduino-compatible microcontroller with sensors to collect measurements from the physical environment.

Potential measurements:

- Temperature
- Humidity
- Light intensity
- Distance
- Acceleration
- Pressure

- Establish hardware/software communication

Create reliable communication between the Arduino and Python.

Conceptually:
Sensors → Arduino → Serial communication → Python

Python should be capable of:

- receiving measurements,
- validating incoming data,
- identifying malformed readings,

- timestamping measurements,
- processing measurements,
- triggering system responses.

- Build a persistent engineering database

Use SQL to store collected data.

Possible tables:

- Sensors
- Measurements
- Experiments
- Alerts
- Devices
- Calibration records

This allows the system to answer questions such as:

- What happened to the temperature over the last six hours?
- When did the system detect an abnormal measurement?

- Create an interactive monitoring interface

Build a GUI that allows the user to:

- see current sensor readings,
- monitor device status,
- view historical measurements,
- inspect alerts,
- select experiments,
- retrieve stored data,
- display graphs.

- Perform quantitative engineering analysis

  Use MATLAB to analyze the collected data.

Potential analyses:

- descriptive statistics
- trends

- regression
- curve fitting
- uncertainty analysis
- filtering
- correlation
- signal analysis

- Systems testing

Test both individual components and the integrated system under standard and nonstandard conditions.
Some potential test cases:

- missing sensor data
- malformed serial messages
- disconnected/simulated unavailable device
- unreasonable sensor values
- repeated measurements
- database errors
- communication interruptions
- threshold violations
- unexpected input

Programming Tools and Roles

Technology	Role
C/C++/Arduino	Microcontroller programming and sensor acquisition
Python	Backend, processing, communication, automation
SQL	Database management and historical data
Python GUI framework	Monitoring dashboard
MATLAB	Engineering analysis and mathematical modeling
GitHub	Version control and documentation
Arduino IDE	Microcontroller development
Serial Communication	Arduino & Python data transfer

Additional Requirements - Core System Functionality

1. Data acquisition

The Arduino periodically measures sensor values.

2. Data validation

Python checks:

- missing values
- unrealistic or unreasonable values
- malformed messages
- sensor communication errors

3. Data storage

Validated measurements are written to SQL. This allows long-term historical analysis.

4. Monitoring system

The GUI displays:

- current values
- sensor status
- recent readings
- historical graphs
- alerts

5. Alert system

Configurable thresholds can be set to determine when a special alert happens.

Research / Engineering Component

***Investigation**: Can historical sensor data be used to detect abnormal environmental behavior earlier than fixed threshold monitoring?*
***Hypothesis**: A historical-data-based anomaly detection approach will identify certain abnormal trends earlier than fixed threshold monitoring while maintaining an acceptable false-alert rate.*

A project report will be made as part of this project component.

Final Deliverables

- System architecture
- Arduino/C++ component
- Python backend
- SQL database
- GUI monitoring dashboard
- MATLAB analysis component
- Automated/integration testing suite
- Experimental dataset
- Research/engineering report
- GitHub repository and documentation
- Limitations and future hardware-extension plan